

Ball World Model

Architecture, Training Objectives, Representation Analysis, and
Translation-Only Evaluation

dr. Jasper Juchem

A technical reference for machine-learning researchers

Idea: RGB sequence $\rightarrow$ spatial features $\rightarrow$ content and motion $\rightarrow$ metric state

Based on the `ball_simulator` source tree and the successful translation-only evaluation
dated 2026-08-27

Version 1.0 – 31 August 2026

Abstract

This report reconstructs the complete translation-only neural state-estimation pipeline implemented in the supplied project source. It explains the tensor geometry, coordinate-aware convolutional encoder, differentiable soft-keypoint bottleneck, content and motion representations, bidirectional latent transition, supervised state heads, iterative metric-space physics corrector, loss design, data protocol, and evaluation methodology. The central empirical observation is a strong representation split: static content is almost perfectly linearly informative about position, while temporal differences and the learned motion representation are strongly informative about velocity. The report distinguishes what is directly guaranteed by the implementation from what is an interpretation supported by probes and interventions. It also clarifies the model’s status as a finite-window neural observer with one-step latent predictive structure rather than an autonomous long-horizon simulator.

Contents

1	Scope, provenance, and terminology	1
2	Problem formulation	1
2.1	Inputs, targets, and deployment contract	1
2.2	Sampling and temporal scale	1
3	Architecture overview	1
4	Coordinate-aware frame encoder	3
4.1	Coordinate augmentation	3
4.2	Convolutional backbone and tensor dimensions	4
4.3	Feature maps versus pixels and global latents	5
5	Soft-keypoint readout and content embedding	6
5.1	Differentiable keypoint statistics	6
5.2	Projection to the content latent	7
6	Motion representation	7
6.1	Feature differences and rate normalization	7
6.2	Spatial motion encoder	7
6.3	Forward and backward motion	8
7	Latent transition and predictive structure	8
8	Metric state heads	8
8.1	Position head	8
8.2	Velocity head	8
8.3	Target normalization and physical units	9
9	Iterative physics corrector	10
9.1	Residual and update	10
9.2	Interpretation and limitations	10
9.3	Could correction occur in latent space?	10
9.4	Energy-based interpretation of iterative refinement	11
10	Training objectives	13
10.1	Why each term matters	13
11	Optimization and implementation configuration	13

12 Evaluation design	13
12.1 Aggregate state accuracy	13
12.2 Linear probes	13
12.3 Interventions	13
13 Results	15
13.1 State-estimation performance	15
13.2 Layer-wise representation probes	15
13.3 Intervention results	15
13.4 Why the results look this way	17
14 Feature maps, keypoints, and Slot Attention	17
15 Ablations and diagnostic experiments	17
16 Recommended architecture extensions	18
16.1 Autonomous prediction	18
16.2 Hybrid latent refinement	18
16.3 Object-centric multi-body extension	19
17 Threats to validity	19
18 Reproducibility checklist	19
19 Conclusion	20
A Tensor-shape reference	20
B Key implementation facts	20

1 Scope, provenance, and terminology

The primary source is the supplied code archive `eventBased_worldModels-state_estimates.zip`, including the `ball_simulator/src/ball_world_model` package and translation experiment configuration. The empirical source is `evaluation_260827.zip`. Numerical values in this document are taken from that evaluation. Literature citations are used to contextualize design patterns, not to substitute for code inspection.

Throughout, B is batch size, T is the number of frames in a context window, $H = W = 256$, $C = 128$ is the final feature-channel count, $K = 8$ is the number of soft keypoints, $D_z = 256$ is the content embedding dimension, and $D_m = 256$ is the motion dimension. A hat denotes a prediction or estimate. Ground-truth metric states are denoted by a superscript `gt`.

Executive model classification. The implemented network is best described as a **visual neural state observer with a one-step joint-embedding latent transition**. It estimates position and velocity from observed image windows. It can predict the next content embedding locally, but it does not yet define a complete autonomous multi-step rollout because the motion representation is extracted from pairs of observed feature maps.

2 Problem formulation

2.1 Inputs, targets, and deployment contract

For each sampled window, the network receives RGB images only:

$$\mathbf{I}_{1:T} \in \mathbb{R}^{B \times T \times 3 \times 256 \times 256}, \quad T = 10. \quad (1)$$

The simulator-provided position and velocity are training and evaluation targets,

$$\mathbf{p}_{1:T}^{\text{gt}} \in \mathbb{R}^{B \times T \times 3}, \quad \mathbf{v}_{1:T}^{\text{gt}} \in \mathbb{R}^{B \times T \times 3}, \quad (2)$$

not inputs to the forward pass. There is no state-label encoder $E_x(\mathbf{p}, \mathbf{v})$ and no target latent computed from numerical ground truth. At deployment, inference is therefore

$$\mathbf{I}_{1:T} \mapsto (\hat{\mathbf{p}}_{1:T}, \hat{\mathbf{v}}_{1:T}). \quad (3)$$

The learned latent variables are visually induced internal coordinates. Metric supervision shapes their usefulness through the readout losses, but does not make them equal to the physical state.

2.2 Sampling and temporal scale

The translation configuration uses rendered 100 Hz video, frame stride one, ten context frames, one prediction frame, and four target-history frames. Thus $\Delta t = 0.01$ s and the ten-frame context spans nine inter-frame intervals, or 90 ms. The dataset is split 80/10/10 at a fixed seed; the data loader uses batch size 16.

3 Architecture overview

The architecture deliberately delays global compression. Each image is first transformed into a spatial tensor $\mathbf{F}_t$, from which two complementary descriptions are derived:

$$\mathbf{z}_t = E_z(\mathbf{F}_t), \quad \text{content / configuration-like information}, \quad (4)$$

$$\mathbf{m}_t = E_m(\mathbf{F}_t, \mathbf{F}_{t+1}, \dot{\mathbf{F}}_t), \quad \text{motion / tangent-like information}. \quad (5)$$

The position head uses $\mathbf{z}_t$, while the velocity head receives the pair $(\mathbf{z}_t, \mathbf{z}_{t+1})$ and the interval motion embedding $\mathbf{m}_t$. A latent transition predicts adjacent content embeddings. A final iterative corrector enforces translational kinematic consistency in metric coordinates.

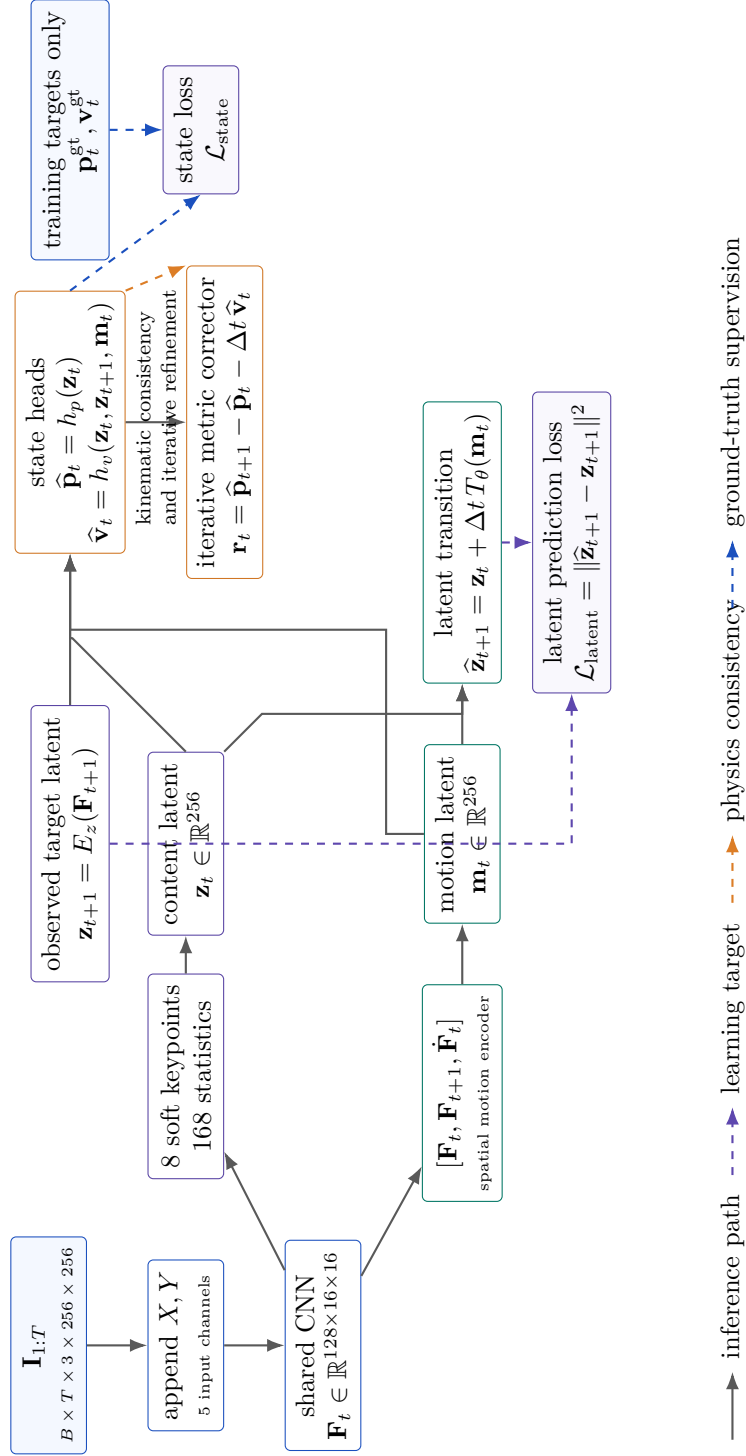


Figure 1: End-to-end computation graph. Solid lines indicate learning targets or consistency constraints. Dashed lines indicate learning targets or consistency constraints.

4 Coordinate-aware frame encoder

4.1 Coordinate augmentation

Two deterministic coordinate planes are appended to every RGB input. For pixel indices (i, j) ,

$$X_{ij} = 2\frac{j}{W-1} - 1, \quad Y_{ij} = 2\frac{i}{H-1} - 1. \quad (6)$$

The backbone input is therefore

$$\tilde{\mathbf{I}}_t = \text{concat}(\mathbf{I}_t, \mathbf{X}, \mathbf{Y}) \in \mathbb{R}^{5 \times 256 \times 256}. \quad (7)$$

Ordinary convolution is translation equivariant and does not expose absolute coordinates directly. Explicit coordinate channels let the network learn translation dependence when the downstream target is Cartesian position, following the CoordConv motivation [2].

Why normalized coordinates are used. The normalized coordinate channels contain the same positional information as integer pixel indices, but express this information using a numerically well-scaled and resolution-independent convention. The transformations

$$X_{ij} = 2\frac{j}{W-1} - 1, \quad Y_{ij} = 2\frac{i}{H-1} - 1 \quad (8)$$

map the discrete indices

$$j \in \{0, \dots, W-1\}, \quad i \in \{0, \dots, H-1\}$$

to the interval $[-1, 1]$. The left and right image boundaries therefore correspond to $X = -1$ and $X = 1$, while the top and bottom boundaries correspond to $Y = -1$ and $Y = 1$. The image centre is approximately $(X, Y) = (0, 0)$. Using raw pixel indices would not provide additional geometric information. For example, the horizontal pixel index can be recovered from the normalized coordinate according to

$$j = \frac{W-1}{2}(X+1). \quad (9)$$

The advantage of normalization is therefore not informational, but numerical and representational. First, the coordinate values remain of order one and are consequently comparable in scale to normalized image intensities. Supplying raw indices between 0 and 255 would introduce coordinate channels with substantially larger numerical magnitude than the normalized RGB channels. Although the first convolution and subsequent normalization layers could compensate for this difference, doing so would create an unnecessary conditioning problem. Second, normalized coordinates express relative image location independently of the chosen pixel resolution. For example, $X = 0.5$ identifies the same relative horizontal position in both a 256-pixel-wide and a 512-pixel-wide image, whereas the corresponding pixel index changes with resolution. This does not make the complete neural network resolution invariant, because object scale, receptive fields, and convolutional sampling still depend on image resolution. It only makes the coordinate convention itself independent of the number of pixels. Third, centring the coordinate range around zero introduces a convenient symmetry:

$$X(-j_{\text{relative}}) = -X(j_{\text{relative}}), \quad Y(-i_{\text{relative}}) = -Y(i_{\text{relative}}).$$

This is useful for spatial regression because locations on opposite sides of the image centre are represented by values of opposite sign. The coordinate channels describe image-plane location and should not be confused with physical world coordinates. The model must still learn the mapping

$$(\text{image location, appearance, temporal context}) \mapsto \text{world position}. \quad (10)$$

When perspective projection is present, image-plane coordinates alone are generally insufficient to infer three-dimensional position. Apparent ball size, surface appearance, environmental geometry, and temporal information may provide additional depth cues.

4.2 Convolutional backbone and tensor dimensions

The backbone is a sequence of four **ConvBlocks**. Each block applies two 3×3 convolutions; the first has the listed stride, and the second has stride one. Each convolution is followed by GroupNorm and SiLU.

Each **ConvBlock** contains two learned two-dimensional convolution operators. Let

$$W_1 \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 3 \times 3} \quad (11)$$

denote the kernel tensor of the first convolution, and let

$$W_2 \in \mathbb{R}^{C_{\text{out}} \times C_{\text{out}} \times 3 \times 3} \quad (12)$$

denote the kernel tensor of the second convolution. The first convolution changes the channel dimension from C_{in} to C_{out} and performs spatial downsampling with stride $s = 2$. The second convolution preserves both the number of channels and the spatial resolution. To make the stride explicit, define

$$\text{Conv}_{3 \times 3, s}(x; W)$$

as a two-dimensional convolution with kernel tensor W , kernel size 3×3 , stride s , and padding one. One block can then be written as

$$h_1 = \text{SiLU}(\text{GN}(\text{Conv}_{3 \times 3, s}(x; W_1))), \quad (13)$$

$$h_2 = \text{SiLU}(\text{GN}(\text{Conv}_{3 \times 3, 1}(h_1; W_2))), \quad (14)$$

$$\text{Block}_s(x) = h_2. \quad (15)$$

Component-wise, the first convolution has the form

$$[\text{Conv}_{3 \times 3, s}(x; W_1)]_{c, i, j} = \sum_{c'=1}^{C_{\text{in}}} \sum_{u=-1}^1 \sum_{v=-1}^1 W_{1, c, c', u, v} \tilde{x}_{c', si+u, sj+v}, \quad (16)$$

where $\tilde{x}$ denotes the zero-padded input. The stride s determines the displacement in input coordinates between neighbouring output activations. In the current backbone, $s = 2$ for the first convolution of every block and $s = 1$ for the second convolution. The convolutional biases are disabled in the implementation because every convolution is followed by GroupNorm. The normalization includes learned affine scale and offset parameters, making a separate convolutional bias largely redundant. The notation in (15) is used instead of the abbreviated operator $W *_s x$. If the latter notation is encountered, $*_s$ should be read as convolution with stride s , rather than scalar multiplication or element-wise multiplication.

Despite an explanatory comment referring to the ResNet idea, the inspected **ConvBlock** is a sequential block and does not add an explicit identity skip connection. This distinction is important when translating comments into an architectural diagram.

Table 1: Exact frame-backbone tensor geometry for a single frame.

Stage	Input channels	Output channels	Stride	Spatial size	Output shape
Coordinate augmentation	3	5	1	256×256	$5 \times 256 \times 256$
ConvBlock 1	5	32	2	128×128	$32 \times 128 \times 128$
ConvBlock 2	32	64	2	64×64	$64 \times 64 \times 64$
ConvBlock 3	64	128	2	32×32	$128 \times 32 \times 32$
ConvBlock 4	128	128	2	16×16	$128 \times 16 \times 16$

Thus

$$\boxed{\mathbf{F}_t \in \mathbb{R}^{128 \times 16 \times 16}}. \quad (17)$$

For full windows, the implementation flattens batch and time for shared frame encoding and reshapes back to

$$\mathbf{F}_{1:T} \in \mathbb{R}^{B \times T \times 128 \times 16 \times 16}. \quad (18)$$

4.3 Feature maps versus pixels and global latents

An RGB pixel channel has a fixed meaning. A feature channel has a learned meaning and can respond to boundaries, markers, object texture, contrast, or combinations of appearance and location. The 16×16 grid preserves explicit spatial organization, unlike a global vector $\mathbf{z}_t \in \mathbb{R}^{256}$. Four stride-two stages produce an effective sampling stride of 16 pixels. Accounting for eight 3×3 convolutions, a final cell has a theoretical receptive field of approximately 91×91 input pixels, so neighboring cells are highly overlapping contextual measurements.

The theoretical receptive field can be calculated recursively. Let r_ℓ denote the receptive-field width after layer ℓ , measured in original input pixels. Let j_ℓ denote the effective jump between the centres of adjacent layer- ℓ activations, again measured in input pixels. At the image input,

$$r_0 = 1, \quad j_0 = 1. \quad (19)$$

For a convolution with kernel width k_ℓ and stride s_ℓ , the update rules are

$$j_\ell = j_{\ell-1}s_\ell, \quad (20)$$

$$r_\ell = r_{\ell-1} + (k_\ell - 1)j_{\ell-1}. \quad (21)$$

The backbone contains eight 3×3 convolutions, two per block. The first convolution in each block has stride two and the second has stride one. Therefore, the stride sequence is

$$(s_1, \dots, s_8) = (2, 1, 2, 1, 2, 1, 2, 1), \quad (22)$$

while $k_\ell = 3$ for all eight layers. Applying (21) gives

Layer	Kernel	Stride	Jump j_ℓ	Receptive field r_ℓ
Input	—	—	1	1
Block 1, convolution 1	3	2	2	3
Block 1, convolution 2	3	1	2	7
Block 2, convolution 1	3	2	4	11
Block 2, convolution 2	3	1	4	19
Block 3, convolution 1	3	2	8	27
Block 3, convolution 2	3	1	8	43
Block 4, convolution 1	3	2	16	59
Block 4, convolution 2	3	1	16	91

The final feature map therefore has

$$j_{\text{final}} = 16, \quad r_{\text{final}} = 91. \quad (23)$$

The value $j_{\text{final}} = 16$ means that the centres of adjacent final feature-map cells are separated by 16 input pixels. The value $r_{\text{final}} = 91$ means that each final cell can theoretically depend on a 91×91 region of the input image. It is therefore incorrect to interpret the 16×16 feature grid as a partition into non-overlapping 16×16 image patches. The stride-one convolutions enlarge the receptive field without reducing the output resolution, so neighbouring final cells summarize strongly overlapping input regions. The value 91×91 is a theoretical receptive field. It identifies all input pixels that can influence an activation through the computational graph. The effective receptive field can be smaller and nonuniform because the learned kernels, nonlinearities, and data distribution determine the actual sensitivity. In practice, input pixels near the centre of the theoretical receptive field often exert more influence than pixels near its boundary. Feature cells near the image boundary also partially depend on the zero-padding introduced by the convolutions.

The feature tensor contains $128 \cdot 16 \cdot 16 = 32768$ scalar activations per frame. It is intentionally richer than the 256-dimensional content latent. This allows temporal differencing and spatial motion estimation to occur before global pooling discards location.

5 Soft-keypoint readout and content embedding

5.1 Differentiable keypoint statistics

The feature encoder produces one spatial tensor per frame,

$$\mathbf{F}_t \in \mathbb{R}^{128 \times 16 \times 16}. \quad (24)$$

A learned 1×1 convolution combines all 128 feature channels and maps them to $K = 8$ heatmap-logit channels:

$$\mathbf{L}_t = \text{Conv}_{1 \times 1}(\mathbf{F}_t; W_{\text{heat}}) \in \mathbb{R}^{K \times 16 \times 16}, \quad K = 8. \quad (25)$$

The model therefore produces eight heatmaps per input frame. It does not produce eight heatmaps per backbone feature channel. Each of the eight output heatmaps is a learned combination of all 128 input feature channels.

For keypoint index $k \in \{1, \dots, K\}$, the spatial logits are normalized using a softmax over all 16×16 locations:

$$\alpha_{t,k,i,j} = \frac{\exp L_{t,k,i,j}}{\sum_{i'=1}^{16} \sum_{j'=1}^{16} \exp L_{t,k,i',j'}}. \quad (26)$$

Consequently,

$$\alpha_{t,k,i,j} \geq 0, \quad \sum_{i,j} \alpha_{t,k,i,j} = 1. \quad (27)$$

Each normalized heatmap can therefore be interpreted as a categorical probability distribution over spatial feature-map locations.

Let

$$\mathbf{x}_{ij} = \begin{bmatrix} X_{ij} \\ Y_{ij} \end{bmatrix} \quad (28)$$

denote the normalized coordinate associated with feature-map location (i, j) . The expected coordinate of heatmap k is

$$\boldsymbol{\mu}_{t,k} = \sum_{i,j} \alpha_{t,k,i,j} \mathbf{x}_{ij}. \quad (29)$$

The vector $\boldsymbol{\mu}_{t,k} \in \mathbb{R}^2$ is called a *soft keypoint*. The term *keypoint* refers to the intended role of the coordinate as a learned spatial landmark. A heatmap may, for example, attend to the projected ball centre, a surface marker, part of the silhouette, a high-contrast edge, or another position-sensitive structure. These meanings are not specified in advance. They emerge only if they help minimize the training objective. The term *soft* distinguishes (29) from a hard spatial maximum. A hard keypoint would be selected by

$$(i_{t,k}^*, j_{t,k}^*) = \arg \max_{i,j} L_{t,k,i,j}. \quad (30)$$

The **argmax** operation selects one discrete grid location and is not differentiable with respect to the heatmap logits. In contrast, the softmax-weighted expectation in (29) changes smoothly when probability mass moves between neighbouring cells. The resulting coordinate can lie between feature-map cell centres and can be optimized by ordinary backpropagation. This operation is also commonly described as a spatial soft-argmax. The spatial variance associated with keypoint k is

$$\sigma_{t,k}^2 = \sum_{i,j} \alpha_{t,k,i,j} (\mathbf{x}_{ij} - \boldsymbol{\mu}_{t,k})^{\odot 2}, \quad (31)$$

where $\odot 2$ denotes component-wise squaring. A concentrated heatmap has low variance, while a broad or multimodal heatmap produces larger variance. The implementation additionally retains a confidence value and a locally weighted appearance vector

$$\mathbf{a}_{t,k} \in \mathbb{R}^{16}. \quad (32)$$

Each keypoint therefore contributes

$$2 \text{ coordinate values} + 2 \text{ variance values} + 1 \text{ confidence value} + 16 \text{ appearance values} = 21 \quad (33)$$

scalars. With $K = 8$, the complete keypoint representation has dimension

$$K(2 + 2 + 1 + 16) = 8 \cdot 21 = 168. \quad (34)$$

The eight keypoints should not automatically be interpreted as eight distinct physical objects. The current scene contains one ball, so several keypoints may describe different landmarks or appearance cues on the same object. Furthermore, unless an explicit diversity or repulsion loss is introduced, the architecture does not guarantee that all heatmaps specialize to different locations. Two heatmaps may overlap, exchange roles, or become useful under different appearance conditions. Their empirical semantics should therefore be evaluated by visualizing all eight heatmaps over representative trajectories.

5.2 Projection to the content latent

The 168-dimensional keypoint descriptor is projected through

$$\mathbb{R}^{168} \xrightarrow{\text{Linear}} \mathbb{R}^{256} \xrightarrow{\text{LayerNorm}} \mathbb{R}^{256} \xrightarrow{\text{SiLU}} \mathbb{R}^{256}, \quad (35)$$

yielding $\mathbf{z}_t \in \mathbb{R}^{256}$.

Interpretation. The soft-keypoint bottleneck is object-aware but is not Slot Attention. Soft keypoints summarize learned landmark heatmaps; Slot Attention uses iterative competitive attention to produce exchangeable object slots that can bind to different entities [3]. For the current single-ball scene, keypoints provide a lower-complexity geometric bottleneck. Multiple interacting objects would motivate slots or another set-valued representation.

6 Motion representation

6.1 Feature differences and rate normalization

For each adjacent frame pair, the raw feature difference and feature rate are

$$\Delta \mathbf{F}_t = \mathbf{F}_{t+1} - \mathbf{F}_t, \quad (36)$$

$$\dot{\mathbf{F}}_t \approx \frac{\mathbf{F}_{t+1} - \mathbf{F}_t}{\Delta t}. \quad (37)$$

A running channel-wise normalizer stabilizes the scale of feature rates. This is especially consequential at 100 Hz: division by $\Delta t = 0.01$ multiplies raw differences by 100, so unnormalized rates can have substantially different optimization scale from the feature activations.

6.2 Spatial motion encoder

The motion encoder receives the spatial concatenation

$$\mathbf{U}_t = \text{concat}(\mathbf{F}_t, \mathbf{F}_{t+1}, \text{norm}(\dot{\mathbf{F}}_t)) \in \mathbb{R}^{384 \times 16 \times 16}. \quad (38)$$

Convolution, GroupNorm, SiLU, pooling, and a linear projection compress this tensor to

$$\mathbf{m}_t \in \mathbb{R}^{256}, \quad t = 1, \dots, T - 1. \quad (39)$$

The essential design choice is that motion is extracted while the representation is still spatial. A moving activation can be distinguished from static background evidence before all positions are mixed into a global vector.

6.3 Forward and backward motion

Motion is calculated in both temporal directions. Reversal changes the interval orientation and should change the sign of translational velocity while preserving speed. The backward branch supports a reversal-consistency objective and provides a direct intervention test: reverse the input sequence and ask whether estimated velocities reverse accordingly.

7 Latent transition and predictive structure

A two-layer MLP maps motion to a content-latent rate:

$$\hat{\mathbf{z}}_t = T_\theta(\mathbf{m}_t). \quad (40)$$

The next latent is predicted with an Euler-style residual step:

$$\hat{\mathbf{z}}_{t+1} = \mathbf{z}_t + \Delta t \hat{\mathbf{z}}_t. \quad (41)$$

The backward counterpart predicts $\mathbf{z}_t$ from $\mathbf{z}_{t+1}$ and backward motion. This is joint-embedding prediction: the target is the embedding computed from the observed adjacent image, not a reconstructed pixel frame. This resembles the broad JEPA principle of predicting target representations rather than pixels [1], but the present model is supervised jointly by metric state labels and is specialized to temporal dynamics.

Why this is not yet a full autonomous world model. The transition predicts one adjacent content embedding, while $\mathbf{m}_t$ itself is produced from two observed feature maps. To roll out without future images, the system would also need dynamics for the motion state, for example $(\mathbf{z}_{t+1}, \mathbf{m}_{t+1}) = G(\mathbf{z}_t, \mathbf{m}_t)$, or an explicit physical rollout built on decoded $(\hat{\mathbf{p}}_t, \hat{\mathbf{v}}_t)$.

8 Metric state heads

8.1 Position head

For translation, the position head maps each frame content latent to three normalized components:

$$\hat{\mathbf{p}}_t^{\text{norm}} = h_p(\mathbf{z}_t), \quad h_p : \mathbb{R}^{256} \rightarrow \mathbb{R}^3. \quad (42)$$

The head consists of Linear, LayerNorm, SiLU, optional dropout, and a final Linear layer.

8.2 Velocity head

An interval representation concatenates both endpoint content embeddings and the motion embedding:

$$\mathbf{q}_t = \text{concat}(\mathbf{z}_t, \mathbf{z}_{t+1}, \mathbf{m}_t) \in \mathbb{R}^{768}. \quad (43)$$

Velocity is decoded by

$$\hat{\mathbf{v}}_t^{\text{norm}} = h_v(\mathbf{q}_t), \quad h_v : \mathbb{R}^{768} \rightarrow \mathbb{R}^3. \quad (44)$$

Pair t describes interval $[t, t + 1]$. To maintain a frame-wise $(B, T, 3)$ output contract, the implementation assigns the final frame the last available interval velocity.

8.3 Target normalization and physical units

The position and velocity heads are trained to predict standardized target values. The normalization quantities

$$\boldsymbol{\mu}_p, \boldsymbol{\sigma}_p, \boldsymbol{\mu}_v, \boldsymbol{\sigma}_v \in \mathbb{R}^3 \quad (45)$$

are component-wise statistics of the training data. They are not learned through gradient descent. Let the training set provide N valid position and velocity samples,

$$\left\{ \mathbf{p}_n^{\text{gt}}, \mathbf{v}_n^{\text{gt}} \right\}_{n=1}^N, \quad \mathbf{p}_n^{\text{gt}}, \mathbf{v}_n^{\text{gt}} \in \mathbb{R}^3.$$

The component-wise position mean is estimated as

$$\boldsymbol{\mu}_p = \frac{1}{N} \sum_{n=1}^N \mathbf{p}_n^{\text{gt}}, \quad (46)$$

and the corresponding component-wise standard deviation is

$$\boldsymbol{\sigma}_p = \sqrt{\frac{1}{N} \sum_{n=1}^N \left(\mathbf{p}_n^{\text{gt}} - \boldsymbol{\mu}_p \right)^{\odot 2}}. \quad (47)$$

Analogously,

$$\boldsymbol{\mu}_v = \frac{1}{N} \sum_{n=1}^N \mathbf{v}_n^{\text{gt}}, \quad (48)$$

$$\boldsymbol{\sigma}_v = \sqrt{\frac{1}{N} \sum_{n=1}^N \left(\mathbf{v}_n^{\text{gt}} - \boldsymbol{\mu}_v \right)^{\odot 2}}. \quad (49)$$

The square root and powers in these expressions are applied component-wise. The implementation may apply a small numerical lower bound to standard deviations to avoid division by values close to zero. During training, physical targets are standardized according to

$$\mathbf{p}_t^{\text{norm}} = \left(\mathbf{p}_t^{\text{gt}} - \boldsymbol{\mu}_p \right) \oslash \boldsymbol{\sigma}_p, \quad (50)$$

$$\mathbf{v}_t^{\text{norm}} = \left(\mathbf{v}_t^{\text{gt}} - \boldsymbol{\mu}_v \right) \oslash \boldsymbol{\sigma}_v, \quad (51)$$

where $\oslash$ denotes component-wise division. Standardization improves the conditioning of the supervised regression problem by placing the position and velocity components on comparable numerical scales. Without standardization, components with a larger numerical range could dominate the mean-squared loss and its gradients. The neural heads produce estimates in this normalized coordinate system:

$$\hat{\mathbf{p}}_t^{\text{norm}}, \quad \hat{\mathbf{v}}_t^{\text{norm}}. \quad (52)$$

Before applying the metric-space physics corrector, these estimates are converted back to physical units:

$$\hat{\mathbf{p}}_t = \boldsymbol{\sigma}_p \odot \hat{\mathbf{p}}_t^{\text{norm}} + \boldsymbol{\mu}_p, \quad (53)$$

$$\hat{\mathbf{v}}_t = \boldsymbol{\sigma}_v \odot \hat{\mathbf{v}}_t^{\text{norm}} + \boldsymbol{\mu}_v, \quad (54)$$

where $\odot$ denotes component-wise multiplication. The training pipeline computes these statistics from the training-data loader before optimizing the network. The resulting vectors are supplied to the model and stored as persistent non-trainable buffers. Consequently, they are included in model checkpoints and move with the model between devices, but they are not updated by the optimizer. The statistics should be estimated using the training split only. Including validation or test trajectories in their calculation would constitute data leakage.

Denormalization must precede metric-space refinement. In normalized coordinates, the expression

$$\hat{\mathbf{p}}_{t+1}^{\text{norm}} - \hat{\mathbf{p}}_t^{\text{norm}} - \Delta t \hat{\mathbf{v}}_t^{\text{norm}} \quad (55)$$

is not generally a valid physical residual because position and velocity use different means and standard deviations. After applying (54), the kinematic residual

$$\mathbf{r}_t = \hat{\mathbf{p}}_{t+1} - \hat{\mathbf{p}}_t - \Delta t \hat{\mathbf{v}}_t \quad (56)$$

is dimensionally consistent:

$$[\mathbf{r}_t] = [\text{m}] - [\text{m}] - [\text{s}][\text{m/s}] = [\text{m}]. \quad (57)$$

9 Iterative physics corrector

9.1 Residual and update

For each adjacent pair, define

$$\mathbf{r}_t^{(k)} = \mathbf{p}_{t+1}^{(k)} - \mathbf{p}_t^{(k)} - \Delta t \mathbf{v}_t^{(k)} \in \mathbb{R}^3. \quad (58)$$

The corrector concatenates the two positions, velocity, and residual:

$$\mathbf{c}_t^{(k)} = [\mathbf{p}_t^{(k)}, \mathbf{p}_{t+1}^{(k)}, \mathbf{v}_t^{(k)}, \mathbf{r}_t^{(k)}] \in \mathbb{R}^{12}. \quad (59)$$

An MLP maps $\mathbb{R}^{12}$ to nine corrections, interpreted as updates to the first position, second position, and interval velocity. With shared parameters across refinement steps,

$$(\mathbf{p}_t^{(k+1)}, \mathbf{p}_{t+1}^{(k+1)}, \mathbf{v}_t^{(k+1)}) = \mathcal{C}_\theta(\mathbf{c}_t^{(k)}). \quad (60)$$

When a frame participates in two intervals, proposed position updates are accumulated and averaged. The corrector is iterated `refinement_iterations` times. Its final layer is initialized to zero, initially preserving the uncorrected estimator and allowing residual corrections to emerge gradually.

9.2 Interpretation and limitations

The corrector is a learned unrolled inference procedure, not an exact projection onto the kinematic constraint. Reusing an MLP for a fixed number of steps does not prove contraction or convergence. Performance for more iterations than used during training should not be assumed.

9.3 Could correction occur in latent space?

A latent residual can be defined as

$$\mathbf{r}_{z,t} = \mathbf{z}_{t+1} - \mathbf{z}_t - \Delta t T_\theta(\mathbf{m}_t). \quad (61)$$

An iterative latent corrector could refine $(\mathbf{z}_t, \mathbf{z}_{t+1}, \mathbf{m}_t)$ before decoding, improving reuse across translation, rotation, and richer tasks. However, latent consistency does not guarantee metric physical consistency: arbitrary invertible distortions, nuisance content, or collapsed dimensions may satisfy a latent equation while decoded states violate mechanics. A robust extension is therefore hybrid:

$$\text{latent refinement} \longrightarrow \text{metric decoding} \longrightarrow \text{lightweight physical projection}. \quad (62)$$

9.4 Energy-based interpretation of iterative refinement

A more structured alternative to an unconstrained residual corrector is to interpret refinement as approximate optimization of a scalar consistency energy. Let

$$\mathbf{s}^{(k)} \quad (63)$$

denote the complete candidate state after refinement iteration k . The superscript (k) denotes the internal correction iteration and should not be confused with physical time, which is indexed by the subscript t . For a metric-space corrector applied to one temporal interval, one possible definition is

$$\mathbf{s}^{(k)} = \begin{bmatrix} \mathbf{p}_t^{(k)} \\ \mathbf{p}_{t+1}^{(k)} \\ \mathbf{v}_t^{(k)} \end{bmatrix} \in \mathbb{R}^9. \quad (64)$$

For a latent-space corrector, the refined variable could instead be

$$\mathbf{s}^{(k)} = \begin{bmatrix} \mathbf{z}_t^{(k)} \\ \mathbf{z}_{t+1}^{(k)} \\ \mathbf{m}_t^{(k)} \end{bmatrix} \in \mathbb{R}^{768}. \quad (65)$$

The feedforward observer provides the initial estimate $\mathbf{s}^{(0)}$. Refinement then produces the sequence

$$\mathbf{s}^{(0)} \longrightarrow \mathbf{s}^{(1)} \longrightarrow \dots \longrightarrow \mathbf{s}^{(K)}. \quad (66)$$

Let

$$E_\theta : \mathbb{R}^{D_s} \longrightarrow \mathbb{R} \quad (67)$$

be a differentiable scalar energy function. A lower value of $E_\theta(\mathbf{s})$ represents a state that is more consistent with the visual observations, learned latent dynamics, and physical constraints. A gradient-based refinement step has the form

$$\mathbf{s}^{(k+1)} = \mathbf{s}^{(k)} - \alpha_k \nabla_{\mathbf{s}} E_\theta(\mathbf{s}^{(k)}), \quad (68)$$

where

$$\nabla_{\mathbf{s}} E_\theta(\mathbf{s}) = \begin{bmatrix} \frac{\partial E_\theta}{\partial s_1} \\ \vdots \\ \frac{\partial E_\theta}{\partial s_{D_s}} \end{bmatrix} \quad (69)$$

is the gradient of the scalar energy with respect to all components of the refined state. The gradient points in the local direction of steepest energy increase. The negative gradient in (68) therefore gives the local direction of steepest energy decrease. The quantity $\alpha_k > 0$ is the refinement step size at iteration k . It controls the magnitude of the correction. A small value produces a conservative update, while an excessively large value can overshoot a local minimum or cause oscillatory behaviour. Possible choices include a fixed step size,

$$\alpha_k = \alpha, \quad (70)$$

a separate learned positive scalar for every unrolled iteration,

$$\alpha_0, \dots, \alpha_{K-1}, \quad (71)$$

or a learned state-dependent step size,

$$\alpha_k = A_\phi(\mathbf{s}^{(k)}). \quad (72)$$

When different components of $\mathbf{s}$ have substantially different scales or curvature, a positive matrix-valued preconditioner may be used:

$$\mathbf{s}^{(k+1)} = \mathbf{s}^{(k)} - \mathbf{P}_k \nabla_{\mathbf{s}} E_{\theta}(\mathbf{s}^{(k)}). \quad (73)$$

For latent refinement, a suitable energy should not contain only the latent transition residual. Such an energy could be minimized by moving toward a dynamically consistent state that is no longer supported by the images. A hybrid energy can instead be decomposed as

$$E_{\theta} = \lambda_{\text{obs}} E_{\text{obs}} + \lambda_z E_{\text{latent}} + \lambda_p E_{\text{physics}} + \lambda_{\text{prior}} E_{\text{prior}}. \quad (74)$$

The observation term anchors corrected latents to the frame-encoder output:

$$E_{\text{obs}} = \|\mathbf{z}_t - \mathbf{z}_t^{\text{enc}}\|_2^2 + \|\mathbf{z}_{t+1} - \mathbf{z}_{t+1}^{\text{enc}}\|_2^2. \quad (75)$$

Here $\mathbf{z}_t^{\text{enc}}$ and $\mathbf{z}_{t+1}^{\text{enc}}$ are the latents obtained directly from the observed images and remain fixed during one refinement procedure. The latent-dynamics term measures disagreement with the learned transition:

$$E_{\text{latent}} = \|\mathbf{z}_{t+1} - \mathbf{z}_t - \Delta t T_{\theta}(\mathbf{m}_t)\|_2^2. \quad (76)$$

The decoded physical term preserves an interpretable translational constraint:

$$E_{\text{physics}} = \|h_p(\mathbf{z}_{t+1}) - h_p(\mathbf{z}_t) - \Delta t h_v(\mathbf{z}_t, \mathbf{z}_{t+1}, \mathbf{m}_t)\|_2^2. \quad (77)$$

A prior term can additionally discourage implausibly large corrections or penalize departure from the feedforward motion estimate:

$$E_{\text{prior}} = \|\mathbf{m}_t - \mathbf{m}_t^{\text{enc}}\|_2^2 + \beta \|\mathbf{s} - \mathbf{s}^{(0)}\|_2^2. \quad (78)$$

This construction makes the semantics of the iteration clearer because every refinement step has the same explicit objective: reduce $E_{\theta}(\mathbf{s})$. The refinement trajectory can be diagnosed by plotting

$$E_{\theta}(\mathbf{s}^{(0)}), \quad E_{\theta}(\mathbf{s}^{(1)}), \quad \dots, \quad E_{\theta}(\mathbf{s}^{(K)}), \quad (79)$$

as well as the individual observation, latent, and physical contributions. By contrast, an unconstrained residual corrector has the generic form

$$\mathbf{s}^{(k+1)} = \mathbf{s}^{(k)} + C_{\theta}(\mathbf{s}^{(k)}). \quad (80)$$

The network directly proposes a correction vector, but the update is not explicitly derived from a scalar objective. Consequently, repeated applications may reduce one inconsistency while increasing another, may oscillate, or may rely specifically on the number of unrolled iterations used during training. The energy-based form does not by itself guarantee global convergence. The learned energy is generally nonconvex, and a large step size may still increase the energy. The formulation does, however, provide a common optimization interpretation and makes convergence properties empirically testable. Appropriate diagnostics include

$$E_{\theta}(\mathbf{s}^{(k+1)}) \leq E_{\theta}(\mathbf{s}^{(k)}), \quad (81)$$

$$\|\nabla_{\mathbf{s}} E_{\theta}(\mathbf{s}^{(k)})\|_2 \rightarrow 0, \quad (82)$$

$$\|\mathbf{s}^{(k+1)} - \mathbf{s}^{(k)}\|_2 \rightarrow 0. \quad (83)$$

These quantities should be evaluated both within the trained number of iterations and beyond it. If additional refinement steps consistently reduce the energy and improve state accuracy, the corrector behaves more like an iterative inference algorithm. If performance is optimal only at the exact unrolled depth used during training, the module is better interpreted as a fixed-depth recurrent neural block.

10 Training objectives

The total objective combines supervised readout, latent prediction, physics consistency, reversal consistency, and anti-collapse regularization. In schematic form,

$$\mathcal{L} = \mathcal{L}_{\text{state}} + \lambda_{\text{latent}}\mathcal{L}_{\text{latent}} + \lambda_{\text{kin}}\mathcal{L}_{\text{kin}} + \lambda_{\text{rev}}\mathcal{L}_{\text{rev}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}}. \quad (84)$$

The experiment configuration sets state weight 1.0 and translation consistency weight 0.1. The implemented training module further reports latent-prediction, kinematic, reversal, and variance terms; the effective weighting used in the analyzed run is summarized in Table 2.

10.1 Why each term matters

The position loss alone could produce an excellent static regressor but provide no useful velocity representation. The velocity loss requires temporal evidence but does not by itself require a coherent content transition. The latent prediction term rewards temporal predictability without reconstructing irrelevant pixels. Kinematic consistency couples two separately decoded state quantities. Reversal consistency injects a symmetry of first-order motion. Variance regularization reduces the risk that predictive learning finds a trivial constant representation.

11 Optimization and implementation configuration

The translation observer configuration uses $D_z = 256$, temporal depth 4, eight keypoints, dropout 0, AdamW-style hyperparameters with learning rate 3×10^{-4} and weight decay 0.05, 75 epochs, mixed bfloat16 precision, gradient clipping at norm 1.0, deterministic training, and logging every ten steps. Images are normalized using ImageNet statistics. The data loader uses eight workers, prefetch factor four, pinned memory, and persistent workers.

12 Evaluation design

12.1 Aggregate state accuracy

The test evaluation contains 1,050 windows and 10,500 evaluated frame states. Reported metrics include per-axis RMSE, MAE, and R^2 for position and velocity, plus aggregate Euclidean errors. The main aggregate values are:

$$\text{RMSE}_{p,\text{mean}} = 1.42 \text{ mm}, \quad \text{RMSE}_{v,\text{mean}} = 32.35 \text{ mm/s}. \quad (85)$$

Position has $R^2 > 0.9999$ on all axes. The weakest velocity component is the y axis with $R^2 = 0.9798$.

12.2 Linear probes

Frozen representation tensors are regressed to position or velocity using linear models. High probe R^2 means the quantity is available through an affine readout, not merely encoded in an arbitrarily nonlinear way. This is a useful diagnostic of coordinate organization, but it is not proof of causal disentanglement.

12.3 Interventions

The evaluator applies temporal interventions: normal order, reversed order, repeated last frame, and shuffled order. These tests separate sensitivity to temporal direction from mere sensitivity to frame content. If velocity were inferred primarily from static appearance, reversing the sequence would not reliably reverse velocity and repeating one frame would not collapse speed.

Table 2: Objective roles and effective weights represented in the analyzed training setup.

Term	Weight	Representative expression	Role
State	1	$\ \hat{\mathbf{p}} - \mathbf{p}^{gt}\ ^2 + \ \hat{\mathbf{v}} - \mathbf{v}^{gt}\ ^2$	Anchors internal representations to metric state.
Latent prediction	0.1	$\ \hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\ ^2$	Encourages predictable temporal organization.
Kinematic consistency	0.01	$\ \mathbf{p}_{t+1} - \mathbf{p}_t - \Delta t \mathbf{v}_t\ ^2$	Connects position and velocity estimates.
Time reversal	0.1	$\ \mathbf{v}_t^{back} + \mathbf{v}_t^{fwd}\ ^2$	Encourages odd velocity under temporal reversal.
Variance floor	0.01	penalty when feature standard deviation is too small	Discourages representational collapse.

13 Results

13.1 State-estimation performance

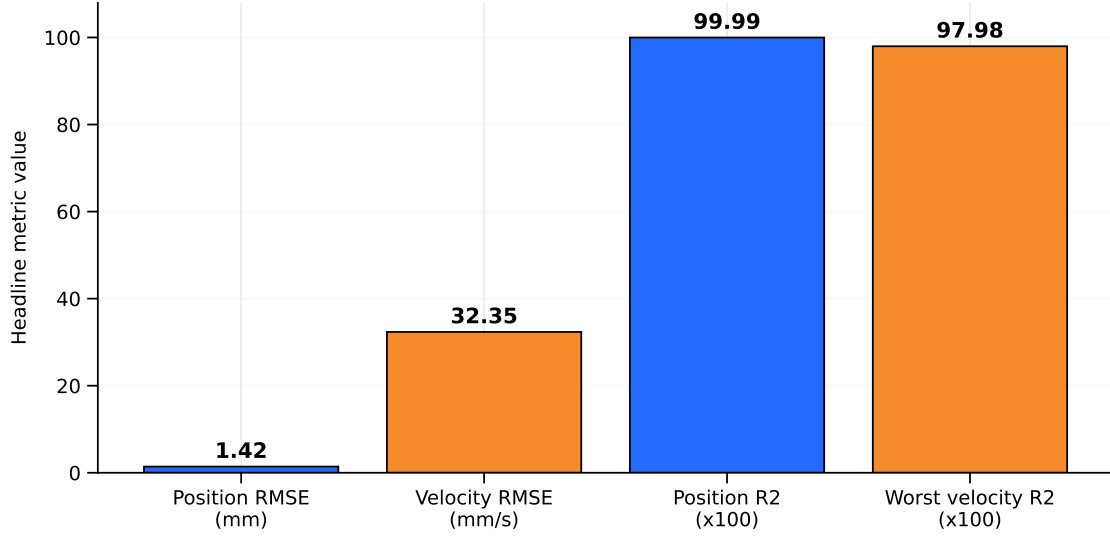


Figure 2: Headline translation-only metrics. Position is essentially exact at the rendered scale; velocity is accurate but has a larger and direction-dependent residual.

The millimetric position error is consistent with a coordinate-aware encoder followed by a keypoint-like spatial bottleneck. Velocity is harder because it depends on differences over a 10 ms interval. Small visual localization errors are divided by a small Δt , amplifying rate noise. A simple propagation argument illustrates the scale:

$$\delta v \approx \frac{\sqrt{2} \delta p}{\Delta t}. \quad (86)$$

With δp at the millimetre level and $\Delta t = 0.01$ s, naive differencing alone could yield errors on the order of tenths of metres per second. The learned spatial motion encoder reduces this substantially, but the principle explains why velocity remains the more difficult output.

13.2 Layer-wise representation probes

The content latent permits almost perfect linear position decoding ($R^2 \approx 0.999$), while the motion-forward mean permits nearly linear velocity decoding ($R^2 \approx 0.996$). The difference between adjacent content embeddings already carries substantial velocity information ($R^2 \approx 0.969$). These observations support, but do not prove, a configuration/tangent interpretation:

$$\mathbf{z}_t \sim \text{configuration-like coordinates}, \quad \mathbf{m}_t \sim \text{tangent-like coordinates}. \quad (87)$$

The spatial-map mean has an effective rank near 1.22 in the evaluation statistics. This should not be read as the rank of the full spatial tensor: spatial averaging can discard precisely the location-dependent variation the encoder was designed to preserve. It may indicate a compact global summary, or it may expose an axis-selective bottleneck. Channel-wise and spatially resolved singular spectra are needed to distinguish these hypotheses.

13.3 Intervention results

The baseline mean predicted speed is approximately 0.645 m/s. Repeating the last frame produces only 0.012 m/s, strong evidence that velocity depends on temporal change. Reversal changes

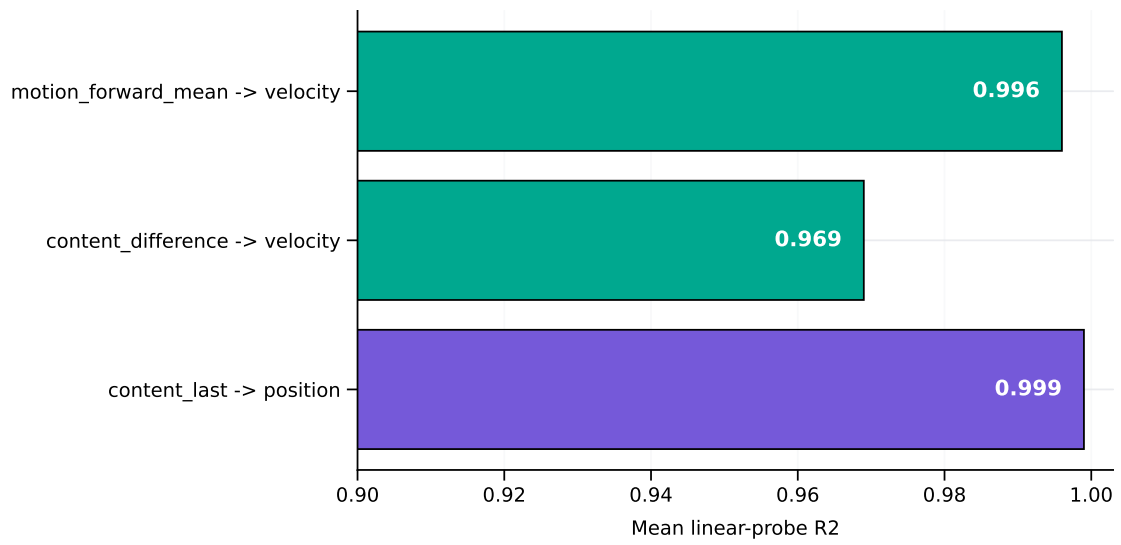


Figure 3: Selected linear-probe results. Values summarize mean R^2 across physical components.

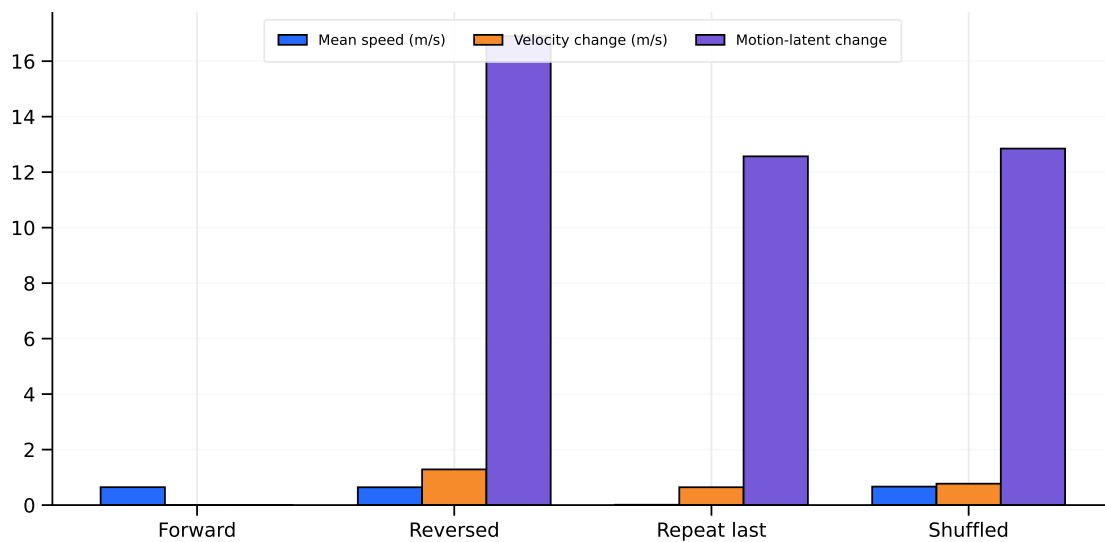


Figure 4: Temporal intervention summary. Repeating a static frame suppresses predicted speed; reversing the sequence significantly changes and approximately reverses velocity.

velocity by approximately 1.286 m/s, close to twice the baseline speed as expected for sign reversal, and the reported reversal error is about 0.062 m/s. Shuffling substantially alters motion latents and velocity while retaining approximately the same speed scale, indicating sensitivity to local temporal order rather than only overall frame content.

13.4 Why the results look this way

1. **Position is easiest because absolute coordinates are explicit.** Coordinate channels expose image location, the feature map maintains spatial organization, and the keypoint readout computes differentiable expected coordinates.
2. **Velocity benefits from pre-pooling temporal structure.** The motion encoder sees $\mathbf{F}_t$, $\mathbf{F}_{t+1}$, and normalized feature rate on a 16×16 grid. It can suppress static background and combine local changes before global compression.
3. **The head assignment encourages factorization.** Position is read from $\mathbf{z}_t$ alone; velocity receives endpoint content and $\mathbf{m}_t$. This asymmetric information route makes a position-heavy content latent and velocity-heavy motion latent an economical solution.
4. **Bidirectionality injects a physical symmetry.** Time reversal encourages velocity sign inversion and makes temporal direction testable.
5. **Metric correction improves coherence but cannot repair all perception errors.** The physical corrector adjusts decoded trajectories locally. It does not alter the visual representation, so systematic visual ambiguities or axis-dependent calibration remain upstream.
6. **Higher-speed cases are less forgiving.** Faster motion causes larger inter-frame displacement and may challenge receptive-field correspondences, heatmap smoothness, and the first-order latent transition.

14 Feature maps, keypoints, and Slot Attention

A feature map is a per-location representation. Soft keypoints transform it into a small set of landmark statistics. Slot Attention instead uses iterative competitive attention to produce exchangeable per-object vectors [3]. The correspondence is summarized as

$$\underbrace{\mathbf{F}_t[i, j]}_{\text{per spatial location}} \longrightarrow \underbrace{\mathbf{k}_t^{(1:K)}}_{\text{per learned landmark}} \quad \text{versus} \quad \underbrace{\mathbf{F}_t[i, j]}_{\text{per spatial location}} \longrightarrow \underbrace{\mathbf{s}_t^{(1:K)}}_{\text{per object slot}}. \quad (88)$$

For a single ball, keypoints are appropriate because several landmarks can jointly estimate pose and appearance without solving object assignment. For multiple balls, variable object counts, or interactions, a slot-based or set-based architecture would provide a more natural entity decomposition.

15 Ablations and diagnostic experiments

The following experiments would most efficiently test the architectural explanations.

Ablation	Question	Expected diagnostic
Remove coordinate channels	Is absolute localization learned from content or background shortcuts?	Position error should rise, especially under camera/background changes.

Ablation	Question	Expected diagnostic
Replace keypoints by global average pooling	Does structured localization explain position precision?	Position probes and metric accuracy should degrade; appearance invariance may improve.
Compute motion from $\mathbf{z}_{t+1} - \mathbf{z}_t$ only	Is spatial pre-pooling differing essential?	Velocity error should increase if spatial correspondences matter.
Remove feature-rate normalization	Is optimization sensitive to $1/\Delta t$ scaling?	Training instability or speed-dependent bias may emerge.
Remove reversal loss	Does bidirectional symmetry cause sign-correct reversal?	Reversal error should worsen more than forward RMSE.
Set refinement iterations to 0, 1, 2, 4, 8	Is the corrector genuinely iterative and stable?	Plot kinematic residual and state RMSE versus iteration.
Remove physics corrector but retain kinematic loss	Separate training regularization from inference refinement.	Reveals whether gains arise in representation or post-processing.
Randomize Δt during training	Does the model learn rate rather than frame displacement?	Better transfer across frame rates would support physical scaling.
Spatially resolved SVD	Is low effective rank an averaging artifact?	Compare spectra of flattened $C \times HW$, channel means, and keypoints.
Background/camera intervention	Is position encoded geometrically or through environmental cues?	Stable performance under texture changes supports genuine object localization.

16 Recommended architecture extensions

16.1 Autonomous prediction

To obtain open-loop rollout, introduce a recurrent dynamic state $\mathbf{s}_t = [\mathbf{z}_t; \mathbf{m}_t]$:

$$\hat{\mathbf{s}}_{t+1} = G_\theta(\mathbf{s}_t, \Delta t), \quad (\hat{\mathbf{p}}_{t+1}, \hat{\mathbf{v}}_{t+1}) = H_\theta(\hat{\mathbf{s}}_{t+1}). \quad (89)$$

For translation-only free flight, an interpretable baseline should always be included:

$$\hat{\mathbf{p}}_{t+k} = \hat{\mathbf{p}}_t + k\Delta t\hat{\mathbf{v}}_t, \quad \hat{\mathbf{v}}_{t+k} = \hat{\mathbf{v}}_t. \quad (90)$$

The learned rollout should be compared against this observer-plus-physics baseline, not only against persistence.

16.2 Hybrid latent refinement

A case-independent corrector should refine both content and motion, then retain a decoded physical residual:

$$(\mathbf{z}^{(k+1)}, \mathbf{m}^{(k+1)}) = \mathcal{C}_z(\mathbf{z}^{(k)}, \mathbf{m}^{(k)}, \mathbf{r}_z^{(k)}), \quad (91)$$

$$(\mathbf{p}, \mathbf{v}) = H(\mathbf{z}^{(K)}, \mathbf{m}^{(K)}), \quad (92)$$

$$\mathbf{r}_p = \mathbf{p}_{t+1} - \mathbf{p}_t - \Delta t\mathbf{v}_t. \quad (93)$$

This retains generality while preserving an interpretable metric anchor.

16.3 Object-centric multi-body extension

For multiple bodies, replace the single global content vector by a set of entity states:

$$\mathcal{S}_t = \{\mathbf{s}_t^1, \dots, \mathbf{s}_t^N\}, \quad (94)$$

and model interactions with attention or a graph network. Slot Attention is one possible front end, but temporal slot identity and collision/contact assignment must be handled explicitly.

17 Threats to validity

- The reported experiment is translation-only and successful; conclusions may not transfer to rotation, combined motion, contacts, occlusion, or real imagery.
- Linear decodability is not causal disentanglement. Correlated factors can be linearly recoverable from the same representation.
- The effective-rank statistic depends strongly on which axes are flattened or averaged.
- One-step latent prediction does not establish stable long-horizon rollout.
- Simulator labels and rendering calibration may make the task easier than real visual state estimation.
- A fixed frame rate allows the network to exploit displacement scale. Frame-rate variation is needed to establish genuine rate awareness.
- The iterative corrector may be trained only for a fixed number of unrolled steps; convergence outside that regime is unproven.

18 Reproducibility checklist

1. Preserve trajectory-level train/validation/test separation and fixed seeds.
2. Record exact normalization statistics for images, feature rates, positions, and velocities.
3. Save both raw and corrected outputs to separate correction effects from observer effects.
4. Report errors by axis, speed, image location, and trajectory.
5. Export feature maps, keypoint heatmaps, keypoint confidence, $\mathbf{z}_t$, $\mathbf{m}_t$, latent residuals, and physical residuals.
6. Evaluate forward, reverse, repeat-last, shuffle, and frame-rate interventions.
7. Report probe train/test separation and regularization to avoid optimistic linear probes.
8. Sweep refinement steps at evaluation while plotting residual monotonicity.
9. Compare against direct CNN-to-state, global-pooling, optical-flow, and observer-plus-analytic-rollout baselines.

19 Conclusion

The architecture’s strong translation-only performance is consistent with a coherent chain of inductive biases. Coordinate augmentation makes absolute position accessible; spatial feature maps preserve localization and motion evidence; soft keypoints create a compact geometric description; separate content and motion pathways encourage a configuration/tangent split; latent prediction organizes adjacent representations; reversal consistency encodes temporal parity; and metric-space refinement reduces kinematic inconsistency. The results support this interpretation through excellent position accuracy, strong velocity accuracy, linearly decodable position and velocity in different representation stages, and appropriate responses to temporal interventions. The model should nevertheless be called a neural observer with local predictive structure until autonomous multi-step rollout is implemented and evaluated.

A Tensor-shape reference

Symbol	Meaning	Shape
I	RGB window	$B \times T \times 3 \times 256 \times 256$
$\tilde{I}$	RGB plus coordinate channels	$B \times T \times 5 \times 256 \times 256$
F	final spatial feature maps	$B \times T \times 128 \times 16 \times 16$
k	soft-keypoint descriptor	$B \times T \times 168$
z	content embeddings	$B \times T \times 256$
ΔF	adjacent feature differences	$B \times (T - 1) \times 128 \times 16 \times 16$
U	motion-encoder spatial input	$B \times (T - 1) \times 384 \times 16 \times 16$
m	interval motion embeddings	$B \times (T - 1) \times 256$
q	velocity-head input	$B \times (T - 1) \times 768$
$\hat{p}$	position estimate	$B \times T \times 3$
$\hat{v}$	frame-wise velocity estimate	$B \times T \times 3$

B Key implementation facts

image: 256 x 256 x 3, ImageNet normalization
coordinate input: 2 channels, each in $[-1, 1]$
context: 10 frames at 100 Hz
backbone channels: 5 -> 32 -> 64 -> 128 -> 128
backbone resolution: 256 -> 128 -> 64 -> 32 -> 16
soft keypoints: 8, with 16 appearance values each
content dimension: 256
motion dimension: 256
velocity input: $[z_t, z_{(t+1)}, m_t] = 768$ dimensions
optimizer config: learning rate $3e-4$, weight decay 0.05
training: 75 epochs, bf16 mixed precision, grad clip 1.0

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF International Conference on Computer Vision*, 2023.

- [2] Rosanne Liu, Joel Lehman, Piero Molino, Felipe Petroski Such, Eric Frank, Alex Sergeev, and Jason Yosinski. An intriguing failing of convolutional neural networks and the coordconv solution. In *Advances in Neural Information Processing Systems*, 2018.
- [3] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems*, 2020.